

Activity 2: Portable C++ Domain Core

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K1, A1

Estimated time: 85 minutes

Objective

Implement exact-cent value objects and a small budget policy as a standalone CMake library.

Deliverable

Compiling C++20 library and domain tests

Tools

C++20, CMake, CTest

Included files

- `starter/CMakeLists.txt`
- `starter/include/BudgetPolicy.hpp`
- `starter/src/BudgetPolicy.cpp`
- `starter/tests/BudgetPolicyTests.cpp`
- `solution/CMakeLists.txt`
- `solution/include/BudgetPolicy.hpp`
- `solution/src/BudgetPolicy.cpp`
- `solution/tests/BudgetPolicyTests.cpp`
- `project.yml` — XcodeGen source of truth
- `scripts/generate-project.sh` — regenerates the shared Xcode project
- `scripts/build-simulator.sh` — resolves an installed iPhone simulator and builds
- `scripts/test.sh` — runs the Swift Testing target and any UI test target
- `solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png` — opaque course app icon
- `solution/PrivacyInfo.xcprivacy` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/02-portable-cpp-domain-core`` and inspect the project with `find starter -maxdepth 3 -type f | sort``. Confirm the CMake file, public header, implementation and test file exist.
2. Create an isolated working copy with `cp -R starter workspace``. All edits for this Activity go under `workspace/``.
3. Open `workspace/include/BudgetPolicy.hpp``. Confirm `Money`` stores `std::int64_t cents`` and that `BudgetPolicy::evaluate`` accepts limit, spent and draft values without an Apple framework type.
4. Open `workspace/src/BudgetPolicy.cpp``. Replace the placeholder behaviour with guard clauses that reject a negative limit, negative spent value and a draft amount less than or equal to zero.
5. Add an over-budget guard: reject when `draft.cents > limit.cents - spent.cents``. For every rejection, return the unchanged remaining amount rather than subtracting the invalid draft.
6. Retain the accepted path as `limit.cents - spent.cents - draft.cents``. Do not convert to `double`` and do not introduce a currency-formatted string in the core.
7. Extend `workspace/tests/BudgetPolicyTests.cpp`` with assertions for valid, zero, negative and over-budget cases. Each branch in the implementation must have at least one named or clearly separated assertion.
8. If CMake is installed, run `cmake -S workspace -B workspace/build``, then `cmake --build workspace/build``, then `ctest --test-dir workspace/build --output-on-failure``. Expect one test target and a 100% pass summary.
9. If CMake is unavailable, run `mkdir -p workspace/build && c++ -std=c++20 -Wall -Wextra -Werror -I workspace/include workspace/src/BudgetPolicy.cpp workspace/tests/BudgetPolicyTests.cpp -o workspace/build/DomainCoreTests``.
10. Run the fallback binary with `workspace/build/DomainCoreTests`` and then `printf '%s\n' $?``. Expect exit status `0`` and no assertion failure.
11. Run `rg -n 'UIKit|SwiftUI|Foundation|NSString|NSArray' workspace/include workspace/src``. Expect no matches; any match means the portable boundary has been violated.
12. Compare the working implementation with `solution/`` only after tests pass. Record the commands and results in `workspace/verification.txt``, then remove only `workspace/build/`` when cleanup is required.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate the complete portable C++20 BudgetPolicy slice in workspace/: CMakeLists.txt, include/BudgetPolicy.hpp, src/BudgetPolicy.cpp, and tests/BudgetPolicyTests.cpp. Preserve Money { std::int64_t cents; } and BudgetPolicy::evaluate(Money limit, Money spent, Money draft) const. Reject negative limit/spent, non-positive draft, overflow, and overspend; preserve remaining cents on rejection. Use no Apple types. Output exact path then complete compilable code for every file, followed by clean CMake/CTest commands and expected 100% pass output. No omitted cases or placeholders.

Review and repair prompt

Review the generated DomainCore for floating-point money, signed overflow, incomplete branch tests, platform imports, API drift, permissive warnings, hidden global state, or tests that mirror implementation without proving outcomes. Return findings first. Repair only causal files, with complete contents. Keep the public signature fixed and rerun clean configure, build, CTest, and the Apple-import scan; do not weaken a failing assertion.

****Protected file scope:**** workspace/CMakeLists.txt; workspace/include/BudgetPolicy.hpp; workspace/src/BudgetPolicy.cpp; workspace/tests/BudgetPolicyTests.cpp

****Expected verification:**** cmake -S workspace -B workspace/build && cmake --build workspace/build && ctest --test-dir workspace/build --output-on-failure; all tests pass and Apple import scan is empty.

Xcode and Simulator verification

1. Run `xcrun simctl list devices available` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run `./scripts/generate-project.sh`, then open the generated `.xcodeproj` in Xcode. Confirm the app and test targets match `project.yml`.
3. Run `./scripts/build-simulator.sh`. Expect `\*\* BUILD SUCCEEDED \*\*` and no signing request because the learner build uses `CODE\_SIGNING\_ALLOWED=NO`.
4. Run `./scripts/test.sh`. Expect `\*\* TEST SUCCEEDED \*\*`. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

CTest reports all domain tests passed and the public header contains no Objective-C or Swift constructs.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

Delete only the local build directory and reconfigure; inspect the first compiler diagnostic, not the last.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove build/; keep source, headers and tests.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.